

## SIMATS ENGINEERING

### ASSESSMENT TOOL 2

**COURSE NAME:** Operating Systems

**COURSE CODE:** CSA-0401

**NAME:** G. Yashmita

**REG NO:** 192511032

#### COURSE OUTCOME COVERED

**CO3:** Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

**Assessment Tool Weightage:** 30%

**QUESTIONS:** 5

**Total Marks:** 50

#### CASE STUDY:

**6.**

Differentiate between **TLB hit** and **TLB miss** in detail. Explain how each case affects memory access time and system efficiency, and illustrate with an example.

**7.**

Explain how **memory protection** is achieved in the system using base/limit registers and access permissions. Discuss the importance of protection in a multi-user environment and explain what happens when a process violates access rules.

**8.**

Describe the concept of **segmentation** in detail. Explain how programs are divided into logical segments such as code, data, and stack, and discuss the benefits of segmentation over other memory management techniques.

**9.**

Explain the **structure and functioning of a segment table**. Describe how logical addresses are translated into physical addresses using segment number and offset.

10.

Compare and contrast **paging and segmentation** in detail. Discuss their differences in terms of memory allocation, fragmentation, flexibility, and performance.

#### RUBRICS FOR CALCULATION

| Criteria                | Weightage | Level 4 – Exemplary (5 Marks)                                    | Level 3 – Proficient (4 Marks)                 | Level 2 – Developing (2–3 Marks)        | Level 1 – Beginning (0–1 Mark)                       |
|-------------------------|-----------|------------------------------------------------------------------|------------------------------------------------|-----------------------------------------|------------------------------------------------------|
| Understanding of Case   | 25        | Thorough understanding; clearly identifies all key issues        | Good understanding with most issues identified | Basic understanding; some issues missed | Poor understanding; problem not identified correctly |
| Application of Concepts | 25        | Effectively applies relevant concepts to analyze the case deeply | Applies appropriate concepts with minor gaps   | Limited application of concepts         | Fails to apply relevant concepts                     |
| Analysis                | 20        | In-depth analysis with strong reasoning and insights             | Good analysis with logical reasoning           | Basic analysis with limited depth       | Weak or no analysis; lacks reasoning                 |
| Solution Design         | 20        | Innovative, feasible solutions with strong justification         | Logical solutions with adequate justification  | Basic solutions with weak justification | Incorrect or no solution provided                    |
| Clarity                 | 10        | Clear, well-structured, and easy to understand                   | Organized and understandable presentation      | Limited clarity and structure           | Poor clarity; difficult to understand                |

## 6. Differentiate between TLB Hit and TLB Miss in Detail

### Introduction

A Translation Lookaside Buffer (TLB) is a small, high-speed cache used with virtual memory and paging. It stores recently used mappings between virtual page numbers and physical frame numbers. The TLB avoids repeated page-table accesses and therefore reduces address-translation time.

### TLB Hit

A TLB hit occurs when the requested page number is already present in the TLB. The frame number can be obtained directly without accessing the page table in main memory.

Sequence: CPU generates address → TLB lookup → page found → frame obtained → data accessed.

Assuming T is TLB access time and M is memory access time:

TLB Hit Time = T + M

### TLB Miss

A TLB miss occurs when the requested page number is absent from the TLB. The system must access the page table in main memory, obtain the frame number, and then access the required data.

Sequence: CPU generates address → TLB lookup → page not found → page-table access → frame obtained → data accessed.

TLB Miss Time = T + 2M

### Detailed Difference

TLB Hit: mapping found; page table normally not accessed; one data-memory access after TLB lookup; lower access time and better performance.

TLB Miss: mapping absent; page table must be accessed; page-table access plus data-memory access; higher access time and lower performance for that reference.

### Example and Calculation

Assume TLB access time = 10 ns and main-memory access time = 100 ns.

Hit time = 10 + 100 = 110 ns.

Miss time = 10 + 100 + 100 = 210 ns.

If hit ratio h = 0.80:

EAT = h(Hit Time) + (1-h)(Miss Time)

EAT = (0.8 × 110) + (0.2 × 210) = 130 ns.

Without TLB, the simplified EAT =  $2 \times 100 = 200$  ns. Hence the TLB reduces EAT from 200 ns to 130 ns in this example.

## Factors Affecting TLB Efficiency

TLB size, hit ratio, program locality, replacement policy and handling of TLB entries during context switches influence performance. A high hit ratio is especially valuable because programs commonly exhibit temporal and spatial locality.

## Analysis

A TLB miss does not necessarily mean a page fault. A TLB miss only means that the translation is not cached in the TLB; the page may still be present in main memory. A page fault is a different event in which the required page is not currently in main memory and must be obtained from secondary storage.

## Conclusion

A TLB hit provides fast page-to-frame translation, whereas a TLB miss adds a page-table access and increases memory access time. A high TLB hit ratio therefore improves system efficiency and is important for high-performance virtual-memory systems.

## 7. Memory Protection Using Base/Limit Registers and Access Permissions

### Introduction

Memory protection prevents a process from accessing memory outside its assigned region or performing operations that are not permitted. It is essential when many processes and users execute concurrently.

### Base and Limit Registers

The base register stores the starting physical address of a process's allocated region. The limit register stores the size of that permitted logical address space.

Valid condition:  $0 \leq \text{Logical Address} < \text{Limit}$

For a valid address:

Physical Address = Base + Logical Address

### Worked Example

Suppose Base = 4000 and Limit = 500. Valid logical addresses are 0–499, corresponding to physical addresses 4000–4499.

For logical address 200:  $200 < 500$ , so access is valid. Physical address =  $4000 + 200 = 4200$ .

For logical address 700:  $700 \geq 500$ , so access is invalid and must be rejected.

## **Access Permissions**

Memory protection can also control the operation allowed on a region. Common permissions are Read (R), Write (W), and Execute (X).

Code: usually Read + Execute.

Data: usually Read + Write.

Stack: usually Read + Write.

Read-only data: Read only.

These permissions prevent, for example, writing into protected code.

## **Importance in Multi-User Systems**

Protection provides process isolation, protects user confidentiality, preserves data integrity, prevents user programs from modifying kernel memory, improves system stability, and allows controlled sharing when explicitly configured.

## **Violation Handling**

If a process generates an address outside its permitted range or performs a prohibited operation, the protection hardware detects the violation and raises a protection exception/fault. Control passes to the operating system. The OS can reject the access, invoke an exception handler, or terminate the offending process according to system policy.

## **Analysis**

Base/limit registers provide simple address-range protection, while access permissions provide finer-grained control over operations. Together they create boundaries between processes and between user space and protected operating-system memory.

## **Conclusion**

Memory protection is fundamental to a secure and reliable multi-user system. Base/limit checks prevent out-of-range access, permissions control allowed operations, and protection faults ensure that violations are handled by the operating system rather than allowing unsafe access.

# **8. Segmentation in Detail**

## **Introduction**

Segmentation divides a program into variable-sized logical units called segments. It reflects the actual logical structure of a program instead of dividing memory only into equal-sized blocks.

## Logical Segments

Typical segments are Code, Data, Stack and Heap. The code segment contains instructions; data contains global/static variables; stack contains function-call information and local variables; heap contains dynamically allocated data.

## Logical Address

A segmented logical address has two parts: <Segment Number, Offset>. The segment number identifies the segment and the offset identifies a location within that segment.

## Segment Table

Each process has a segment table containing information such as base address, limit and protection.

Example:

Segment 0: Base 1000, Limit 500, Code

Segment 1: Base 3000, Limit 800, Data

Segment 2: Base 5000, Limit 400, Stack

## Address Translation Example

For logical address <1,250>, segment 1 has Base = 3000 and Limit = 800. Since  $250 < 800$ , the address is valid.

Physical Address = Base + Offset =  $3000 + 250 = 3250$ .

If the offset were 900, it would violate the limit and access would be denied.

## Benefits

Segmentation provides logical organization, segment-level protection, controlled sharing, modularity and independent growth of logical components. Code can be protected as read/execute while data can be read/write.

## Comparison with Paging

Paging uses fixed-size pages and focuses on physical-memory allocation. Segmentation uses variable-size logical units and focuses on program structure. Paging mainly avoids external fragmentation, while segmentation can suffer from external fragmentation.

## Limitation

Variable-sized segments can create external fragmentation because free memory becomes divided into holes. Compaction may be required in some systems, adding overhead.

## Conclusion

Segmentation provides a logical, flexible and protection-friendly memory model. It is especially useful when code, data, stack and other modules need to be managed separately, although external fragmentation is its major limitation.

## 9. Structure and Functioning of a Segment Table

### Introduction

A segment table is the data structure used to translate segmented logical addresses into physical addresses. Each process has segment information describing the location, size and protection of its logical segments.

### Logical Address

A logical address is written as <Segment Number, Offset>. The segment number selects a segment-table entry, while the offset identifies the position within that segment.

### Structure

A segment-table entry commonly contains Segment Number, Base Address, Limit and Protection information. Base is the starting physical address; Limit is the segment size or maximum valid offset.

### Example Table

Segment 0: Base 1000, Limit 500, Read+Execute.

Segment 1: Base 3000, Limit 800, Read+Write.

Segment 2: Base 5000, Limit 400, Read+Write.

### Translation Steps

1. CPU generates <segment, offset>.
2. Segment number selects the segment-table entry.
3. Base and limit are obtained.
4. Offset is checked against the limit.
5. Permissions are checked for the requested operation.
6. If valid, physical address = Base + Offset.
7. If invalid, a protection fault is generated.

### Worked Example

For logical address <1,250>, segment 1 has Base 3000 and Limit 800.

$250 < 800$ , so it is valid.

Physical Address =  $3000 + 250 = 3250$ .

### Invalid Example

For logical address <1,900>, segment 1 has Limit 800. Since  $900 \geq 800$ , the offset is invalid. No valid physical address is produced and the access is rejected through a protection fault.

### Importance

Segment tables support address translation, variable-sized segments, segment-level protection, logical sharing and isolation of code, data and stack.

## **Conclusion**

The segment table is central to segmentation. It converts a segment number into base/limit information, checks the offset and permissions, and then forms the physical address as  $\text{Base} + \text{Offset}$ . Thus, it performs both translation and an important protection function.

## **10. Compare and Contrast Paging and Segmentation in Detail**

### **Introduction**

Paging and segmentation are memory-management techniques that translate logical addresses to physical memory, but they divide memory differently. Paging uses fixed-size blocks, whereas segmentation uses variable-size logical program components.

### **Paging**

Logical memory is divided into fixed-size pages and physical memory into equal-size frames. Pages can be placed in any available frames, so contiguous physical allocation is unnecessary. A logical address contains page number and offset.

### **Segmentation**

A program is divided into variable-size logical segments such as code, data, stack and heap. A logical address contains segment number and offset. The segment table provides base and limit information.

### **Memory Allocation**

Paging allows non-contiguous allocation and is efficient when free frames are scattered. Basic segmentation requires suitable contiguous space for each variable-sized segment, making allocation more complex.

### **Fragmentation**

Paging mainly experiences internal fragmentation because a final page may not be completely filled. Segmentation mainly experiences external fragmentation because variable-sized segments leave holes between allocations.

### **Flexibility**

Paging is flexible for physical-memory management and virtual-memory systems. Segmentation is flexible for logical organization because each program component can have a different size and protection policy.

### **Protection and Sharing**

Segmentation naturally supports module-level protection and sharing. Paging can also provide protection using per-page permission bits, but a logical module may span multiple pages.



## **Performance**

Paging works particularly well with virtual memory and page replacement. A TLB can cache page-to-frame mappings and reduce translation overhead. Segmentation requires segment-table lookup and bounds checking and may suffer from allocation/fragmentation overhead.

## **Worked Example**

Suppose a program contains 400 bytes of code, 300 bytes of data and 200 bytes of stack. Segmentation can represent these directly as three logical segments of those sizes. With a page size of 100 bytes, paging represents them as 4 code pages, 3 data pages and 2 stack pages. Those pages may occupy different physical frames.

## **Detailed Comparison**

Paging: fixed-size, page table, non-contiguous physical allocation, mainly internal fragmentation, excellent for virtual memory.

Segmentation: variable-size, segment table, logical program organization, mainly external fragmentation, strong logical protection and sharing.

## **Analysis and Suitability**

Paging is generally preferred when the primary requirements are efficient physical-memory allocation, virtual memory and simple fixed-size replacement. Segmentation is valuable when logical organization, module-level protection and sharing are important. Systems can also combine ideas from both approaches.

## **Conclusion**

Paging and segmentation are complementary approaches rather than universally superior alternatives. Paging emphasizes efficient physical-memory management, while segmentation emphasizes logical structure and protection. The appropriate technique depends on the system's memory, performance, protection and application requirements.